Structures in C

Definition

- Structure is a **user-defined data type in C** which allows you to combine **different data types** to store a particular type of record.
- It is somewhat similar to an Array. The only difference is that array is used to store collection of similar data types while structure can store collection of any type of data.

Example:

Structure is used to represent a record.

- Suppose you want to store record of **Student** which consists of student name, address, roll number and age. You can define a structure to hold this information.
- Suppose you want to keep track of your books in a library. You might want to track the following attributes about each book −

- Title
- Author
- Subject
- Book ID

Defining a structure

- **struct** keyword is used to define a structure.
- **struct** defines a new data type which is a collection of different types of data.
- **Syntax :**

```
struct structure_name
{
 //declaration of different data types
};
```

- **Note:** The closing braces in the structure type declaration must be followed by a semicolon(;).

Example of Structure

```
struct Book
{
 char name[15];
 int price;
 int pages;
};
```

•Here the **struct Book** declares a structure to hold the details of book which consists of three data fields, namely *name*, *price* and *pages*.

•These fields are called **structure elements or members**.

•Each member can have different data type,like in this case, **name** is of char type and **price** is of int type etc.

•**Book** is the name of the structure and is called structure

Declaring Structure Variables

- It is possible to declare variables of a **structure**, after the structure is defined.
- **Structure** variable declaration is similar to the declaration of variables of any other data types.
- Structure variables can be declared in following two ways.

1) Declaring Structure variables separately

```
struct Student
{
 char[20] name;
 int age;
 int rollno;
} ;

struct Student S1 , S2;   //declaring variables of Student
```

2) Declaring Structure Variables with Structure definition

```
struct student
{
 char[20] name;
 int age;
 int rollno;
} S1, S2 ;
```

Accessing Structure Members

- Structure members can be accessed and assigned values in number of ways.
- Structure member has no meaning independently.
- In order to assign a value to a structure member, the member name must be linked with the **structure** variable using dot . operator also called **period** or **member access** operator.

```
struct Book
{
 char name[15];
 int price;
 int pages;
} b1 , b2 ;

b1.price=200;      //b1 is variable of Book type and price is member of Book
```

We can also use scanf() to give values to structure members through

```
scanf(" %s ", b1.name);
scanf(" %d ", &b1.price);
```

Structure Initialization

- Like any other data type, structure variable can also be initialized at compile time.

```
struct Patient
{
 float height;
 int weight;
 int age;
};

struct Patient p1 = { 180.75 , 73, 23 };    //initialization
```

Or

```
struct patient p1;
p1.height = 180.75;     //initialization of each member separately
p1.weight = 73;
p1.age = 23;
```

Array of Structure

- We can also declare an array of **structure**. Each element of the array represents a **structure** variable.
- **Example :** struct employee emp[5];

The below code define an array **emp** of size 5 elements. Each element of array **emp** is of type **employee**

```
#include<stdio.h>
#include<conio.h>
struct employee
{
 char ename[10];
 int sal;
};

struct employee emp[5];
int i,j;
void ask()
{
 for(i=0;i<3;i++)
 {
  printf("\nEnter %dst employee record\n",i+1);
  printf("\nEmployee name\t");
  scanf("%s",emp[i].ename);
  printf("\nEnter employee salary\t");
  scanf("%d",&emp[i].sal);
 }
 printf("\nDisplaying Employee record\n");
 for(i=0;i<3;i++)
 {
  printf("\nEmployee name is %s",emp[i].ename);
  printf("\nSlary is %d",emp[i].sal);
 }
}
void main()
{
 clrscr();
 ask();
 getch();
}
```

Nested Structures

- Nesting of structures, is also permitted in C language.
- **Example :**

```
struct student
{
 char[30] name;
 int age;
   struct address
    {
     char[50] locality;
     char[50] city;
     int pincode;
    };
};
```

Structure as function arguments

- We can pass a structure as a function argument in similar way as we pass any other variable or array
- **Example:**

```
#include<stdio.h>
#include<conio.h>
struct student
{
 char name[10];
 int roll;
};
void show(struct student st);
void main()
{
 struct student std;
 clrscr();
 printf("\nEnter student record\n");
 printf("\nstudent name\t");
 scanf("%s",std.name);
 printf("\nEnter student roll\t");
 scanf("%d",&std.roll);
 show(std);
 getch();
}

void show(struct student st)
{
 printf("\nstudent name is %s",st.name);
 printf("\nroll is %d",st.roll);
}
```